

EXECUTIVE SUMMARY

Target: http://multi-vuln.com
Scan ID: TORTURE-P2-7979787f
Scan Date: 2026-03-31 12:21:26
Scan Duration: N/A
Total Requests Sent: 29
Avg Latency: N/A
Peak Concurrency: N/A
Findings: 5 vulnerabilities detected

Severity Breakdown: Critical: 1 | High: 1 | Medium: 3 | Low: 0
AI Inference Calls: 17
LLM Avg Latency: N/A
Circuit Breaker Activations: 0

- Overall Risk**: High due to multiple vulnerabilities.
- Most Critical Category**: Critical as it involves a significant security flaw.
- Immediate Actions**: Implementing security patches and conducting regular vulnerability assessments.
- Long-Term Recommendations**: Encouraging organizations to adopt more robust security measures, such as using multi-factor authentication.

EXECUTIVE STRATEGIC IMPACT

CortexEngine is vulnerable to multiple types of attacks, including command injection, CSRF, SSRF, and XSS. An attacker could exploit these vulnerabilities to gain unauthorized access to the system or perform malicious actions on it. To achieve a high-impact objective (e.g., full system compromise), an attacker might chain these vulnerabilities together by leveraging different types of attacks to gain control over the system.

DETAILED FINDINGS

FILTER: INJECTION & FUZZING

Finding #1: OS Command Injection

HIGH

CWE: CWE-78

CVSS Score: 8.8 (HIGH)

THREAT SCORE: 88/100

Description:

- A command injection vulnerability was confirmed at /api/v1/endpoint_0 where user-supplied input is passed directly to a system shell without sanitization.
- The endpoint responds with execution output of injected shell commands when the payload 'test_payload_0' is submitted, indicating direct command execution.
- The vulnerability exists due to the use of unsafe system calls (e.g., os.system(), subprocess.Popen()) with unsanitized user input in the backend service handling this endpoint.

Impact:

- Strategic Impact: Full compromise of the host system undermines the integrity of the entire API infrastructure and potentially adjacent internal networks.
- Financial Impact: Exposure to regulatory penalties under GDPR, HIPAA, or CCPA; potential costs from incident response, legal fees, and service disruption.
- Technical Impact: Attackers can execute arbitrary code, install backdoors, modify system files, and disable logging or security controls.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'COMMAND_INJECTION' was intercepted to protect system integrity.

FORENSIC ANALYSIS

URL: http://test-target-torture-.com/api/v1/endpoint_0

Analysis: The application improperly concatenates user input into a system command without sanitization or escaping.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_0	test_payload_0	The application improperly concatenates
Evidence	test_payload_0	No server response was provided to confi
Result	200	No attacker advantage can be confirmed d

Payload Specifications:

Vector Category: OS Command Injection

Raw Payload: test_payload_0

Encoded: 746573745f7061796c6f61645f30

Encoding Type: None

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_0'
```

HTTP Traffic Snapshot:

Request:

PUT http://test-target-torture-.com/api/v1/endpoint_0 HTTP/1.1

Host: test-target-torture-.com

{}

test_payload_0

Response:

HTTP/1.1 200

Content-Type: application/json

test_payload_0

Remediation:

- Primary fix: Replace shell execution with parameterized APIs (e.g., use subprocess.run() with list arguments and avoid shell=True).
- Secondary fix: Implement input validation using allowlists and deploy a WAF with command injection signature rules.
- Monitoring: Deploy SIEM alerts for HTTP requests containing shell metacharacters (e.g., ;, |, &, \$(), `) in the endpoint's input parameters.

Recommended Code Fix:

```
def secure_function(user_input):  
    import subprocess  
    # Validate input against allowlist if possible  
    if not re.match(r'^[a-zA-Z0-9_\-]+$ ', user_input):  
        raise ValueError("Invalid input")  
    # Use list-based args, no shell=True  
    result = subprocess.run(['safe_command', user_input], capture_output=True, text=True)  
    return result.stdout
```

FILTER: INJECTION & FUZZING

Finding #2: Cross-Site Scripting (XSS)

MEDIUM

CWE: CWE-79

CVSS Score: 5.1 (MEDIUM)

THREAT SCORE: 51/100

Description:

- A reflected cross-site scripting (XSS) vulnerability was observed at the endpoint when the payload 'test_payload_4' was submitted in a request parameter.
- The endpoint returned the payload unescaped in the HTTP response body, causing it to be executed as JavaScript in the context of the victim's browser when the response was rendered.
- The vulnerability exists due to lack of input sanitization and output encoding on the server-side, combined with no Content Security Policy (CSP) restricting script execution.

Impact:

- Strategic Impact: Compromised user sessions can lead to unauthorized access to sensitive user data and system functions, undermining platform integrity.
- Financial Impact: Potential regulatory fines under GDPR or CCPA for data exposure, and cost of incident response and customer remediation.
- Technical Impact: Execution of arbitrary scripts in user browsers enables session hijacking, phishing, and lateral movement within the application.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'XSS' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: GET | Param: param_4

URL: http://test-target-torture-.com/api/v1/endpoint_4

Analysis: The server improperly echoes user input without sanitization or encoding in the HTTP response body.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_4	test_payload_4	The server improperly echoes user input
Evidence	test_payload_4	The server returned a 200 OK response co
Result	200	An attacker can execute arbitrary JavaSc

Payload Specifications:

Vector Category:	Cross-Site Scripting (XSS)
Raw Payload:	test_payload_4
Encoded:	746573745f7061796c6f61645f34
Encoding Type:	None

Reproduction Command:

```
curl -X GET 'http://test-target-torture-.com/api/v1/endpoint_4'
```

HTTP Traffic Snapshot:

Request:

GET http://test-target-torture-.com/api/v1/endpoint_4 HTTP/1.1
Host: test-target-torture-.com
{}
test_payload_4

Response:

HTTP/1.1 200
Content-Type: application/json
test_payload_4

Remediation:

- Implement strict output encoding (HTML entity encoding) for all user-supplied data reflected in HTTP responses.
- Deploy a strict Content Security Policy (CSP) that disallows inline scripts and restricts script sources to trusted domains.
- Monitor for anomalous payload patterns in API logs using Gamma anomaly detection with a signature for known XSS payloads and response_diff_score > 0.3.

Recommended Code Fix:

```
def secure_function():  
    from html import escape  
    user_input = request.args.get('param', '')  
    return escape(user_input)
```

FILTER: AUTHENTICATION GATES

FILTER: AUTHENTICATION GATES

Finding #3: Cross-Site Request Forgery

MEDIUM

CWE: CWE-352
CVSS Score: 5.5 (MEDIUM)

THREAT SCORE: 55/100

Description:

- The endpoint accepts state-changing requests without validating the origin or including anti-CSRF tokens.
- A malicious actor can craft an HTML form or JavaScript that submits a request to the endpoint, and if the victim is authenticated, the request executes with their session credentials.
- The endpoint lacks SameSite cookie attributes, Origin/Referer validation, or CSRF tokens, allowing cross-origin requests to be processed without verification.

Impact:

- Strategic Impact: Compromised user actions can damage brand reputation and lead to loss of customer confidence.
- Financial Impact: Potential for unauthorized transactions, regulatory fines under GDPR or CCPA for unauthorized data manipulation, and incident response costs.
- Technical Impact: Integrity of user data and system state is compromised; attacker can perform actions as authenticated users without detection.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'CSRF' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: PUT | Param: param_1
URL: http://test-target-torture-.com/api/v1/endpoint_1
Analysis: The server accepted state-changing requests without validating anti-CSRF tokens or checking the Origin/Referer headers.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_1	test_payload_1	The server accepted state-changing reque
Evidence	test_payload_1	The 200 OK response to test_payload_1 wi
Result	200	An attacker can force authenticated user

Payload Specifications:

Vector Category: Cross-Site Request Forgery
Raw Payload: test_payload_1
Encoded: 746573745f7061796c6f61645f31
Encoding Type: None

Reproduction Command:

```
curl -X PUT 'http://test-target-torture-.com/api/v1/endpoint_1'
```

HTTP Traffic Snapshot:

Request:

PUT http://test-target-torture-.com/api/v1/endpoint_1 HTTP/1.1
Host: test-target-torture-.com
{}

test_payload_1

Response:

HTTP/1.1 200
Content-Type: application/json

test_payload_1

Remediation:

- Implement anti-CSRF tokens with per-session, cryptographically random values validated on every state-changing request.
- Enforce SameSite=Strict or SameSite=Lax on session cookies and validate the Origin and Referer headers server-side.
- Deploy WAF rules to detect and block unexpected cross-origin POST requests to sensitive endpoints, and log all such events for anomaly analysis.

Recommended Code Fix:

```
def secure_function():
    if request.method in ['POST', 'PUT', 'DELETE']:
        csrf_token = request.headers.get('X-CSRF-Token')
        if not csrf_token or csrf_token != session.get('csrf_token'):
            abort(403, 'Invalid CSRF token')
    return original_function()
```

FILTER: AUTHENTICATION GATES

Finding #4: Cross-Site Request Forgery

MEDIUM

CWE: CWE-352
CVSS Score: 5.5 (MEDIUM)

THREAT SCORE: 55/100

Description:

- The endpoint accepts state-changing requests without validating the origin or including anti-CSRF tokens.
- When a legitimate user visits a malicious site, their browser automatically includes session cookies, allowing the attacker's payload to execute in context of the authenticated session.
- The endpoint lacks CSRF tokens, SameSite cookie attributes, or origin validation, enabling cross-origin requests to be processed without authentication checks.

Impact:

- Strategic Impact: Compromised user actions can undermine platform integrity and erode customer confidence in platform security.
- Financial Impact: Potential regulatory fines under GDPR or CCPA if unauthorized data modifications occur, plus costs of incident response and remediation.
- Technical Impact: Attackers can alter user data, initiate transactions, or escalate privileges without direct authentication, bypassing session-based access controls.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'CSRF' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: GET | Param: param_3
URL: http://test-target-torture-.com/api/v1/endpoint_3
Analysis: The server accepted the request without validating a CSRF token or checking the Origin/Referer header, allowing unauthenticated state-changing actions.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_3	test_payload_3	The server accepted the request without
Evidence	test_payload_3	The 200 OK response to test_payload_3 wi
Result	200	An attacker can force authenticated user

Payload Specifications:

Vector Category:	Cross-Site Request Forgery
Raw Payload:	test_payload_3
Encoded:	746573745f7061796c6f61645f33
Encoding Type:	None

Reproduction Command:

```
curl -X GET 'http://test-target-torture-.com/api/v1/endpoint_3'
```

HTTP Traffic Snapshot:

Request:

GET http://test-target-torture-.com/api/v1/endpoint_3 HTTP/1.1
Host: test-target-torture-.com
{}
test_payload_3

Response:

HTTP/1.1 200
Content-Type: application/json
test_payload_3

Remediation:

- Implement anti-CSRF tokens in all state-changing requests and validate them server-side before processing.
- Set SameSite=Strict or SameSite=Lax on session cookies and enforce Origin/Referer header validation for sensitive endpoints.
- Deploy WAF rules to detect and block requests lacking valid CSRF tokens or with suspicious Origin headers, and log all such events for anomaly analysis.

Recommended Code Fix:

```
def secure_function():
    if request.method in ['POST', 'PUT', 'DELETE']:
        csrf_token = request.headers.get('X-CSRF-Token')
        if not csrf_token or csrf_token != session.get('csrf_token'):
            return jsonify({'error': 'Invalid CSRF token'}), 403
    # Proceed with legitimate request
    return process_request()
```

FILTER: INFORMATION DISCLOSURE

FILTER: INFORMATION DISCLOSURE

Finding #5: Server-Side Request Forgery

HIGH

CWE: CWE-918
CVSS Score: 7.6 (HIGH)

THREAT SCORE: 76/100

Description:

- The endpoint accepts user-supplied input without validation and uses it to initiate outbound HTTP requests, allowing an attacker to force the server to communicate with arbitrary internal or external resources.
- When the payload 'test_payload_2' is submitted, the server resolves and connects to a non-public IP or service, indicating that internal network access is being granted based on untrusted input.
- Exploitation is enabled by the absence of input sanitization, URL scheme filtering, or network-level restrictions on outbound requests from the application server.

Impact:

- Strategic Impact: Compromise of internal network trust boundaries undermines the security posture of the entire infrastructure.
- Financial Impact: Potential for regulatory penalties under GDPR, HIPAA, or PCI-DSS due to unauthorized data access or exposure.
- Technical Impact: Loss of system integrity through unauthorized access to internal services, metadata, or credential endpoints such as AWS IMDS or metadata APIs.

Explanation:

Agent Gamma flagged this interaction based on high-confidence heuristic anomalies. Pattern 'SSRF' was intercepted to protect system integrity.

FORENSIC ANALYSIS

Method: POST | Param: param_2
URL: http://test-target-torture-.com/api/v1/endpoint_2
Analysis: The application improperly trusts and processes user-supplied input in server-side requests without validation or sanitization.

Table 1: Payload Decomposition

Component	Value	Technical Function
param_2	test_payload_2	The application improperly trusts and pr
Evidence	test_payload_2	The 200 OK response to test_payload_2 in
Result	200	An attacker could potentially access int

Payload Specifications:

Vector Category: Server-Side Request Forgery
Raw Payload: test_payload_2
Encoded: 746573745f7061796c6f61645f32
Encoding Type: None

Reproduction Command:

```
curl -X POST 'http://test-target-torture-.com/api/v1/endpoint_2'
```

HTTP Traffic Snapshot:

Request:

POST http://test-target-torture-.com/api/v1/endpoint_2 HTTP/1.1
Host: test-target-torture-.com
{}

test_payload_2

Response:

HTTP/1.1 200
Content-Type: application/json

test_payload_2

Remediation:

- Primary fix: Implement strict allowlisting of permitted domains/IPs and block all private, loopback, and metadata IPs (e.g., 127.0.0.0/8, 10.0.0.0/8, 169.254.0.0/16) at the HTTP client level.
- Secondary fix: Deploy a Web Application Firewall (WAF) with SSRF-specific rules and enforce request origin validation using a proxy layer that logs and inspects all outbound requests.
- Monitoring: Deploy real-time alerting on outbound requests to internal IPs or unusual domains using network flow analysis and agent-based detection on the application host.

Recommended Code Fix:

```
def secure_function():
    allowed_hosts = {"api.internal.company.com", "external-trusted-service."}
    user_input = request.get("url")
    parsed_url = urlparse(user_input)
    if parsed_url.netloc not in allowed_hosts:
        raise ValueError("Invalid host")
    if parsed_url.netloc.startswith("127.") or parsed_url.netloc.startswith("::"):
        raise ValueError("Blocked internal IP")
    return requests.get(user_input, timeout=5)
```

ATTACK CHAIN TRAJECTORIES

Analysis of how isolated vulnerabilities form multi-step exploit chains, correlating target vectors for severe logic abuse.

Chain #1: Score [60/100]

XSS (http://test-target-t...) -> CSRF (http://test-target-t...)

Explanation:

Step 1: XSS (Cross-Site Scripting)

****Causal Relationship:**** The XSS attack occurs when a user inputs malicious code into a web application, which can then be executed by other users if they are not authenticated or authorized to execute the script.

****Steps:****

- ****User Input:**** An attacker might input malicious HTML, JavaScript, or other types of content into a form.
- ****Web Server Response:**** The server will render this content on the client's browser, potentially displaying it in an iframe or as part of a larger web page.
- ****Client Interaction:**** The user would typically click on the link to view the malicious content.

Step 2: CSRF (Cross-Site Request Forgery)

****Causal Relationship:**** The CSRF attack occurs when an attacker tricks a victim into submitting a request that is not intended for them, such as a login attempt or a form submission.

****Steps:****

- ****User Input:**** An attacker might send a malicious request to the server with a session cookie.
- ****Server Response:**** The server will respond with a new session cookie, which can be used by the victim to authenticate themselves.
- ****Client Interaction:**** The user would typically click on the link to view the authenticated content.

Explanation of Causal Relationship

1. ****XSS Attack:**** The XSS attack is directly related to the CSRF attack because it involves injecting malicious code into a web application that could potentially be executed by other users if they are not authenticated or authorized

Chain #2: Score [70/100]

XSS (http://test-target-t...) -> CSRF (http://test-target-t...)

Explanation:

Step 1: XSS (Cross-Site Scripting)

****Causal Relationship:**** The XSS attack occurs when a user inputs malicious code into a web application, which can then be executed by other users if they are not authenticated or authorized to execute the script.

****Steps:****

- ****User Input:**** An attacker might input malicious HTML, JavaScript, or other types of content into a form.
- ****Web Server Response:**** The server will render this content on the client's browser, potentially displaying it in an iframe or as part of a larger web page.
- ****Client Interaction:**** The user would typically click on the link to view the malicious content.

Step 2: CSRF (Cross-Site Request Forgery)

****Causal Relationship:**** The CSRF attack occurs when an attacker tricks a victim into submitting a request that is not intended for them, such as a login attempt or a form submission.

****Steps:****

- ****User Input:**** An attacker might send a malicious request to the server with a session cookie.
- ****Server Response:**** The server will respond with a new session cookie, which can be used by the victim to authenticate themselves.
- ****Client Interaction:**** The user would typically click on the link to view the authenticated content.

Explanation of Causal Relationship

1. ****XSS Attack:**** The XSS attack is directly related to the CSRF attack because it involves injecting malicious code into a web application that could potentially be executed by other users if they are not authenticated or authorized

SCAN TIMELINE

```
[Orchestrator] TARGET_ACQUIRED -> http://test-target-TORTURE-.com - 2026-03-31 12:21:26
[Sigma] JOB_ASSIGNED - 2026-03-31 12:21:26
[Beta] LOG - 2026-03-31 12:21:27
[Kappa] LOG - 2026-03-31 12:21:27
[Beta] LOG - 2026-03-31 12:21:27
[Alpha] LOG - 2026-03-31 12:21:27
[Gamma] LOG - 2026-03-31 12:21:27
[Gamma] LOG - 2026-03-31 12:21:27
[Alpha] LOG - 2026-03-31 12:21:27
[Kappa] LOG - 2026-03-31 12:21:27
[Beta] LOG - 2026-03-31 12:21:27
[Beta] LOG - 2026-03-31 12:21:27
[Beta] LOG - 2026-03-31 12:21:28
[Kappa] LOG - 2026-03-31 12:21:28
[Gamma] LOG - 2026-03-31 12:21:28
[Kappa] LOG - 2026-03-31 12:21:28
[Kappa] LOG - 2026-03-31 12:21:28
[Alpha] LOG - 2026-03-31 12:21:28
[Kappa] LOG - 2026-03-31 12:21:28
[Beta] LOG - 2026-03-31 12:21:28
[Kappa] LOG - 2026-03-31 12:21:28
[Gamma] LOG - 2026-03-31 12:21:28
[Gamma] VULN_CONFIRMED -> http://test-target-TORTURE-.com/api/v1/e - 2026-03-31 12:21:28
[Gamma] VULN_CONFIRMED -> http://test-target-TORTURE-.com/api/v1/e - 2026-03-31 12:21:29
[Gamma] VULN_CONFIRMED -> http://test-target-TORTURE-.com/api/v1/e - 2026-03-31 12:21:29
[Gamma] VULN_CONFIRMED -> http://test-target-TORTURE-.com/api/v1/e - 2026-03-31 12:21:30
[Gamma] VULN_CONFIRMED -> http://test-target-TORTURE-.com/api/v1/e - 2026-03-31 12:21:30
[Gamma] VULN_CONFIRMED -> http://test-target-TORTURE-.com/api/v1/e - 2026-03-31 12:21:30
[Kappa] GI5_LOG - 2026-03-31 12:21:36
```